

Quantum Key Distribution

QIC mini-project

Gustav C. R. Groenborg

June 1, 2026

1 Introduction

The key objective of this mini-project is to implement the **BB84**-algorithm as described in [1] and understand how the algorithm works.

1.1 The fundamentals of quantum key distribution

Quantum key distribution is provably secure [1]. This security relies upon fundamental quantum physics such as the no cloning theorem, entanglement and measurement [2]. By example of the **BB84**-algorithm, suppose that Alice and Bob has each their own qubit. They may communicate over two channels, a quantum channel and a classical channel. Eve may eavesdrop, by measuring qubit sent through the quantum channel, however to gain any information, Eve must interact with the system and any interaction with the system will perturb the system [3] which causes errors that will be detected by Alice and Bob [2].

1.2 BB84 protocol

The following describes the BB84-protocol as detailed by [3] and [1]. For the BB84-protocol, suppose Alice and Bob are connected by two channels:

1. A quantum channel
2. A classical channel

An eavesdropper, from now on referred to as Eve, may actively intercept messages on the quantum channel, but may only passively eavesdrop the classical channel. Alice prepares two random strings $a, b \in \{0, 1\}^{4n}$ where n is the desired length of the final key produced by the protocol. Alice then prepares a set of $4n$ qubits, where each qubit is rotated, based on the bits in a and b in accordance to the function described in table 1. Alice then transmits the prepared qubits to Bob over a quantum channel. Upon receiving the qubits, Bob measures each qubit in either the X or Z-basis at random, while recording what basis was used for each measurement in a string c . After having measured all of the received qubits, Bob announces to Alice, that he has received the transmitted qubits.

Data bits	Base bits	Resulting state
1	0	$ 1\rangle$
0	0	$ 0\rangle$
1	1	$ -\rangle$
0	1	$ +\rangle$

Table 1: The encoding scheme of the data bits as described by [1].

Alice now announces b over the classic channel. After constructing the string $c' = \{bit \mid bit \in a \cap c\}$, Bob announces the string of discarded bits, $\{bit \mid bit \notin a \cap b\}$ and Alice then discards appropriately achieving the strings $b' = c'$ and a' is the string resulting from pruning the indices of a that was pruned in b . By all likelihood both Alice and Bob are now left with strings all of length $2n$.

Alice selects the first n elements of a' and announces them to Bob, who then compares them to his own first n elements of c' , if Eve was not eavesdropping on the quantum channel, the error should be 0. If, on the other hand, Eve was eavesdropping, the error will be around $\frac{1}{4}$.

1.2.1 Encoding scheme of the data bits

As described in [1] the data bits are encoded based on the base bits, the encoding scheme is shown in Table 1. As the initial state of each qubit is $|0\rangle$ different quantum gates must be used to achieve the target state of the encoding scheme. To achieve the desired states the Pauli-X gate and the Hadamard gate, H, were used as shown in equations eqs. (1) to (4).

$$|0\rangle = |0\rangle \quad (1)$$

$$|1\rangle = X |0\rangle \quad (2)$$

$$|-\rangle = H X |0\rangle \quad (3)$$

$$|+\rangle = H |0\rangle \quad (4)$$

1.2.2 Measuring the qubits

Upon receiving the qubits, each must be measured in either basis X or Z, but as all qubits are measured in Z on IBM's quantum computers[4], qubits that are supposed to be measured in X must be rotated before measurement. To achieve this You must apply the Hadamard gate before measuring the qubit to obtain the result, as if it were measured in the X-basis. Suppose an arbitrary qubit $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$, if we were to measure $|\psi\rangle$ in the X-basis the probability of $|+\rangle$ would be $\frac{\alpha}{2} + \frac{\beta}{2}$ as per eq. (5). Instead, measuring in the Z-basis the probability of ψ would be $\frac{\alpha}{2}$ as per eq. (6).

$$\begin{aligned}
p_+(|\psi\rangle) &= \langle\psi| E_+ |\psi\rangle \\
&= \langle\psi|+\rangle \langle+|\psi\rangle \\
&= (\alpha \quad \beta) \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \langle+|\psi\rangle \\
&= \frac{\alpha}{2} + \frac{\beta}{2}
\end{aligned} \tag{5}$$

$$\begin{aligned}
p_0(|\psi\rangle) &= \langle\psi| E_0 |\psi\rangle \\
&= \langle\psi|0\rangle \langle 0|\psi\rangle \\
&= (\alpha \quad \beta) \begin{pmatrix} \frac{1}{\sqrt{2}} \\ 0 \end{pmatrix} \langle 0|\psi\rangle \\
&= \frac{\alpha}{2}
\end{aligned} \tag{6}$$

Applying the Hadamard gate to $|\psi\rangle$ before measuring in the Z-basis will yield the same result as when measured in the X-basis, this can be seen from eq. (7).

$$\begin{aligned}
p_0(H|\psi\rangle) &= \langle\psi| H E_0 H |\psi\rangle \\
&= \langle\psi| H |0\rangle \langle 0| H |\psi\rangle \\
&= \langle\psi|+\rangle \langle+|\psi\rangle && \text{by eq. (4)} \\
&= \frac{\alpha}{2} + \frac{\beta}{2} && \text{by eq. (5)}
\end{aligned} \tag{7}$$

1.2.3 Why is it safe?

It must be made clear that the BB84-protocol as implemented in section 3 is not inherently safe. As can be concluded from the results described in section 4 bits from the key are leaked to the eavesdropper. However, eavesdropping on the quantum channel does cause a detectable increase in the error rate, as is shown in section 4 and described in [1] and [3].

If the quantum channel is eavesdropped, Eve must make measurements in order to gain any information about the transmitted quantum states [3]. By the no-cloning theorem, Eve cannot clone the transmitted states forcing her to reconstruct the states herself after measurement by preparing new quantum states as Alice did and then transmitting them to Bob [3, 1].

Half of the time Eve will guess the correct basis, resulting in the same bit for all three, but the other half of the time Eve will guess the wrong basis, leaving Bob's bit to match Alice's only half of the time. As the odds of Eve guessing the right basis and Eve guessing the wrong basis are mutually exclusive events, the probabilities of each scenario must be added in order to obtain the rate of agreement between Alice and Bob in the case of eavesdropping, $(\frac{1}{2} \cdot 1) + (\frac{1}{2})^2 = \frac{3}{4}$ [3]. This property means that eavesdropping can be detected, when the agreement between Alice and Bob is not sufficiently high.

2 State of the art

The recent advances, in both security proofs and flaws of Quantum Key Distribution, QKD, are covered in [5]. While stating that the security guarantees of QKD are much stronger than classical key distribution, as it relies on the fundamental properties of quantum mechanics rather than the computational power of your adversary, it also highlights the potential weaknesses [5]. While there has been further developments in protocols both deriving from and improving upon the BB84-protocol, all of them are vulnerable to the hardware side-channels attacks [5].

Both [5] and [6] mention Measurement Device Independent QKD, MDI-QKD, as a potential candidate to mitigating detector side-channel attacks, however MDI-QKD is closer related to the E91-protocol and relies on the Bell inequalities to prove non-interference of an active adversary [5, 1]. Furthermore, [6] notes Twin-Field QKD, TD-QKD, as a more recent breakthrough allowing for greater bit rates than more traditional QKD protocols and MDI-QKD.

The dependency on flawless physical systems in the security proofs of QKD is also highlighted in [7], which further states that QKD protocols derived from the BB84-protocol are usually more feasible to implement in practical applications, though they lack the stronger security guarantees of the protocols derived from the E91-protocol.

3 Implementation

For the implementation, the protocol described by Chuang in [1, p. 588], was closely followed, in order to ensure that the protocol was correct. The protocol was implemented in Python¹ using the Qiskit Library[8]. To ease experimenting a CLI was built around the algorithm allowing for setting different parameters that the experiment should run under. Amongst these parameters is `--with-eve`, which enables eavesdropping on the communication between Alice and Bob. A communicating actor was implemented as a class `Person`, which utilised the dispatch pattern, listing 1, to determine which action to take upon receiving a message and inspecting its header.

Upon starting the program, Alice, will prepare and “transmit” her quantum states, where after she, through the classical channel, will request Bob to confirm that he received the quantum states. When preparing the states, a `QuantumCircuit` object, line 3 listing 2, with equal number of quantum registers and classical registers is instantiated. The states are then prepared by applying quantum gates to each qubit of the circuit based on the bits and bases of the instance calling the method.

After the states has been prepared, they are then “transmitted” to Bob. Au lieu an actual transmission of qubits, Alice, transfer the constructed `QuantumCircuit`

¹The source code is available at the following GitHub repository: <https://github.com/GustavGroenborg/BB84-protocol>

```

1 def _handle_message(self, msg: Message):
2     match msg.header:
3         case Message.Header.ABORT:
4             # Retry not implemented
5             return False
6         case Message.Header.BASES:
7             discarded_bases = self._read_bases_and_discard(msg.content)
8             self._announce_discarded_bases(discarded_bases)
9             return True
10        case Message.Header.BIT_CHECK:
11            test_bits = self._check_bits(msg.content)
12            self._announce_test_bits(test_bits)
13            return True
14        case Message.Header.CONFIRM_OK:
15            return self._handle_confirm_ok(msg.content)
16        case Message.Header.DISCARDED_BITS:
17            self._discard_bits_and_bases(msg.content)
18            self._announce_bit_check()
19            return True
20        case Message.Header.REQUEST_OK:
21            return self._handle_request_ok(msg.content)
22        case Message.Header.TEST_BITS:
23            loss = self._compare_test_bits(msg.content)
24            self._announce_test_bit_loss(loss)
25            return True
26    return False

```

Listing 1: The method responsible for handling the messages received through the classic channel and deciding upon an appropriate action. Please note that error handling and logging has been removed from the included code.

```

1 def _prepare_states(self):
2     num_qubits = len(self.bases)
3     bb84_circuit = QuantumCircuit(num_qubits, num_qubits)
4     for n in range(num_qubits):
5         match (self.bits[n], self.bases[n]):
6             case (1, 0):
7                 bb84_circuit.x(n)
8             case (0, 0):
9                 pass
10            case (1, 1):
11                bb84_circuit.x(n)
12                bb84_circuit.h(n)
13            case (0, 1):
14                bb84_circuit.h(n)
15    return bb84_circuit

```

Listing 2: The method responsible for “preparing the states for transmission”. Please note that error handling and logging has been omitted from this snippet.

object through the socket, line 3 listing 3, to Bob, who then compiles, runs and measures the circuit. After the transmission, Alice requests through the classical channel, lines 4-6 listing 3, that Bob has received the quantum states.

```

1 def transmit_states(self):
2     bb84_circuit = self._prepare_states()
3     self._socket.quantum_channel.transmit(bb84_circuit)
4     self._announce_classic(
5         Message.Header.REQUEST_OK,
6         "states received")

```

Listing 3: Method, that allowed for “transmission of qubits” from one instance of the `Person` class to another.

Upon collecting the message through the classic channel, the dispatch method handler will call the method shown in listing 4. Upon receiving the `QuantumCircuit` object, line 2 listing 4, Bob will add measurements to the quantum circuit as dictated by his randomly generated measurement base string. However, as evident from line 6 listing 4, qubits supposed to be measured in the X-basis are rotated by the Hadamard gate, before being measured in the Z=basis. After defining all the measurements, the quantum circuit is run, line 8 listing 4, and the results are collected in the following line. In the last line of listing 4 the result is *processed*, by reversing the endian of the resulting bit string, and assigning the bit string to Bob’s `self.bits` variable.

```

1 def _receive_and_measure(self):
2     bb84_circuit = self._socket.quantum_channel.receive()
3     num_qubits = bb84_circuit.num_qubits
4     for n in range(num_qubits):
5         if self.bases[n] == MeasurementBase.X:
6             bb84_circuit.h(n) # Flipped due to hardware limitations
7             bb84_circuit.measure(n, n)
8     result = self._run_circuit(bb84_circuit, self._setup)
9     self._process_result(result)

```

Listing 4: The method responsible for “receiving and measuring the states”. The hardware limitation mentioned on line 6 is briefly explained in section 1.2.2. Please note that error handling and logging has been omitted for this snippet.

The method responsible for running a circuit is described in listing 5. Before actually running the circuit, it is transpiled, if run by the Aer simulator² or on actual quantum hardware. The produced quantum circuit is transpiled in order to ensure minimal noise and adherence to the limitations of the quantum hardware that it is to be run on [10]. Lines 4-6, listing 5, sets up the transpilation

²The Qiskit Aer simulator is a simulator that provides realistic noise models for quantum circuits produced by Qiskit. More information about the Aer simulator is available at its GitHub repository [9]

pass manager for running on actual quantum hardware, whereas lines 8-11, listing 5, sets up the transpilation pass manager for running on the Aer simulator. Lastly, if the option `--noiseless` is passed to the CLI, the circuit is neither run on actual quantum hardware or with realistic simulation, but rather idealistic simulation of a noiseless quantum computer [9].

```

1 def _run_circuit(qc: QuantumCircuit, setup: Dict[str, Any]):
2     if setup["optimise"]:
3         if setup["service"] is not None:
4             preset_pass_manager = generate_preset_pass_manager(
5                 target=setup["backend"].target,
6                 optimization_level=setup["optimise"])
7         else:
8             preset_pass_manager = generate_preset_pass_manager(
9                 optimization_level=setup["optimise"])
10            optimised_circuit = preset_pass_manager.run(qc)
11            job = setup["sampler"].run([optimised_circuit], shots=1)
12        else:
13            job = setup["sampler"].run([qc], shots=1)
14    return job.result()

```

Listing 5: The static method responsible for running the circuit, either on a simulator or on actual quantum hardware.

After having run the circuit and processed the results, Bob announces to Alice, that he has received the states. Alice then announces the bases in which she used to encode her qubits. Upon receiving Alice’s bases, Bob compares them one by one with his own bases, and then either appends his corresponding bit of the matching base, line 4 listing 6, to his key `self.key` or discard the base and its corresponding bit, line 7 listing 6, if there is no match. After having discarded all bases that did not match those of Alice’s, Bob announces the indices of the discarded bases to Alice, through the classical channel.

Upon receiving the indices of the bits to be discarded, Alice discards the

```

1 def _read_bases_and_discard(self, incoming_bases: List[int]):
2     discarded_indices = [ ]
3     for n in range(len(incoming_bases)):
4         if self.bases[n] == incoming_bases[n]:
5             self.key.append(self.bits[n])
6         else:
7             discarded_indices.append(n)
8     self._discard_bits_and_bases(discarded_indices)
9     return discarded_indices

```

Listing 6: The method responsible for reading the bases received over the classical channel and discarding the bases, and corresponding bits, that does not match the received bases. The function `_discard_bits_and_bases` in line 8 trivially removes each index in the passed list, from `self.bits` and `self.bases`.

```

1 def _announce_bit_check(self):
2     self.key = self.bits
3     mid = len(self.bits) // 2
4     self.test_bits = [bit for i, bit in enumerate(self.key) if i in
        ↪ range(mid)]
5     self._discard_key_indices(list(range(mid)))
6     self._announce_classic(
7         Message.Header.BIT_CHECK,
8         list(range(mid)))

```

Listing 7: The method responsible for announcing the bit check on the classic channel. The method `_discard_key_indices` on lines 5, trivially discards the passed indices of `self.key`.

appropriate bits and bases. Alice then prepares for the bit check, by first copying her current bits into her empty key, line 2 listing 7. The first half of the key is then used as the bits used to check for any error between her and Bob's bits, lines 3-4 listing 7, because these bits will be indirectly leaked through later announcements on the classic channel, they are discarded from Alice's key. Lastly, the indices that are used for the test bits are announced over the classic channel, such that Bob knows which bits to discard from his key. Upon receiving the announcement of the indices of the test bits Bob constructs the list of test bits, from his own key, based on the received indices, and then announces these test bits on the classic channel to Alice, lines 11-12 listing 1.

After receiving the test bits Alice compares them to her own test bits, lines 3-5 listing 8, and records the number of matches. After every bit has been compared she announces the loss to Bob, requesting him to confirm or deny the loss.

```

1 def _compare_test_bits(self, incoming_bits: List[int]) -> float:
2     matches = 0
3     for (a, b) in zip(self.test_bits, incoming_bits):
4         if a == b:
5             matches += 1
6     loss = 1 - matches / len(self.test_bits)
7     return loss

```

Listing 8: This method compares the received test bits with the test bits of the calling object.

When Bob receives Alice's request announcement, he parses the message for the float value of the announced loss, lines 2-4 listing 9, if the loss is below a predetermined threshold, he responds with a `CONFIRM_OK` announcement, over the classic channel, confirming that the loss rate was within the accepted threshold. If the loss instead was above the accepted threshold, he instead announces to Alice that the protocol should be aborted. In either case, this is end of the protocol as it was implemented.

```

1 def _confirm_loss(self, msg_content: str):
2     match = re.search(r"(\w+):\s*([\d.]+)", msg_content)
3     key = match.group(1)
4     value = float(match.group(2))
5     if key == "loss" and value < MAX_LOSS:
6         self._announce_classic(
7             Message.Header.CONFIRM_OK,
8             msg_content)
9         return True
10    else:
11        self._announce_classic(
12            Message.Header.ABORT,
13            msg_content)
14        return False

```

Listing 9: The method responsible for determining whether or not the loss is above a predetermined threshold. Please note that error handling and logging has been removed from the code shown in the listing.

3.1 Implementation of eavesdropping

The eavesdropper is implemented as a deriving class, `Eavesdropper` of the `Person` class that Alice and Bob is instantiated as. The main difference in implementation lies in the reading on the classic channel. The eavesdropper, from now on referred to as Eve, only passively reads the messages announced on the classic channel. Furthermore, Eve mimics every action that Bob does, in an attempt to gain as much information as possible about the key that Alice and Bob are trying to construct. As Alice, Bob and Eve are each run on their own thread, communicating through the socket handled on the main thread, the spin interval of Eve is to half of that of Alice and Bob. This ensures that Eve reads the messages on the classic channel, before Bob reads a message from the classic channel. The output of a run of the implementation, with eavesdropping, on an simulated ideal quantum computer can be seen in appendix A.

4 Execution

In order to ensure portability with regards to different backends, optimisation levels and whether the CLI is run online or offline two setup functions were created, listings 10 and 11. When the parameter `--online` is passed to the CLI, the function `setup_quantum_computer`, listing 10, is called. The function creates a Python dictionary, containing the necessary information to run implemented quantum circuit on actual quantum hardware. Please note that the IBM token and CRN instance token, must either be passed by the command lines parameters `--ibm-token` and `--crn`, or by the environment variables `IBM.TOKEN` and `CRN`.

If instead it is desired that the implemented quantum circuit is run offline

```

1 def setup_quantum_computer(args):
2     service = QiskitRuntimeService(
3         channel="ibm_quantum_platform",
4         token=args.ibm_token,
5         instance=args.crn
6     )
7     backend = service.backend(args.qpu, instance=args.crn)
8     sampler = SamplerV2(backend)
9     return {
10         "service": service,
11         "backend": backend,
12         "sampler": sampler,
13         "optimise": args.optimise
14     }

```

Listing 10: The function creating the setup in order to ensure that the circuit can be run on a quantum computer on a given backend that is supported by the Qiskit SDK. The formal parameter `args` is the parsed command line arguments.

instead, no parameter needs to be passed to the CLI, as this is the default mode of running. Upon startup, the function `setup_simulator`, listing 11 is called, ensuring the necessary setup for running the implemented quantum circuit on a local simulator. Please note, that if the command line parameter `--noiseless` is passed, no noise model is setup, and the implemented quantum circuit is run on an simulation of an ideal quantum computer

```

1 def setup_simulator(args):
2     if args.noiseless is True:
3         backend_sim = AerSimulator()
4         sampler = BackendSamplerV2(backend=backend_sim)
5     else:
6         fake_backend = FakeFez()
7         noise_model = NoiseModel.from_backend(fake_backend)
8         backend_sim = AerSimulator(noise_model=noise_model)
9         sampler = BackendSamplerV2(backend=backend_sim)
10    return {
11        "service": None,
12        "backend": backend_sim,
13        "sampler": sampler,
14        "optimise": args.optimise
15    }

```

Listing 11: How the setup for running a quantum circuit on a given fake backend, either with a noise model or without a noise model, that is supported by the Aer simulator. The formal parameter `args` is identical to the one in listing 10

Please note that due to hardware limitations of the classical computer, on which the simulations were run, the key lengths differ based on the scenario.

The average key lengths across all scenarios are presented in table 2. The key length is presented as an average, as they might differ by up to one bit in length, due to a changing number of common bases between Alice and Bob.

	Noiseless offline	Noisy offline	Online
Without Eve	32.30	7.20	32.10
With Eve	32.80	7.20	32.00

Table 2: Average sifted key length in bits, obtained across runs of the implemented protocol with different configurations, both with and without eavesdropping.

5 Discussion

The implementation of the BB84 did not provide results that were outside of the expected range. As can be concluded from figs. 1 and 3, the simulated ideal quantum computer performs remarkably similar to the actual quantum hardware that the circuit was run on, when considering the run without eavesdropping. The mean deviation of the results from simulated noiseless quantum computer is 0.0, whereas the mean deviation of the results from the actual quantum computer is 0.0093.

	Noiseless offline		Noisy offline		Online	
	<i>fidelity</i>	<i>deviation</i>	<i>fidelity</i>	<i>deviation</i>	<i>fidelity</i>	<i>deviation</i>
Alice & Bob	0.7542	0.0518	0.7734	0.0926	0.7638	0.0536
Eve & Alice	0.7681	0.0648	0.7742	0.0789	0.7712	0.0535
Eve & Bob	0.7359	0.0541	0.7745	0.0950	0.7118	0.0692

Table 3: The mean fidelity and mean deviation obtained on different runs of the implemented protocol with eavesdropping enabled.

Presented in table 3, the mean fidelity of both the results produced by the noiseless simulator, the noisy simulator and the actual quantum hardware lies within a range of 0.0627.

The outlier, achieving 1.0 fidelity between Eve and Bob, fig. 2, was only once out of all test runs ever done, regardless of the setup. The cause of this outlier, might be the pseudo random number generator used to generate the bits and bases of the implemented protocol, not being truly random. Another source of error is the key length of 7 bits in this test scenario yielding the probability of Eve guessing the right key by pure luck at a obtainable $7.8215 \cdot 10^{-3}$ when comparing to the probability with key length of 32 bits being less than $2.328 \cdot 10^{-10}$.

As shown in table 4, the results obtained when running the implemented protocol without eavesdropping demonstrates lower error than the runs with eavesdropping. These results support the claim that it is possible to detect when the system is eavesdropped on. However, it should also be noted that the eavesdropping detection can only be guaranteed across multiple runs of the

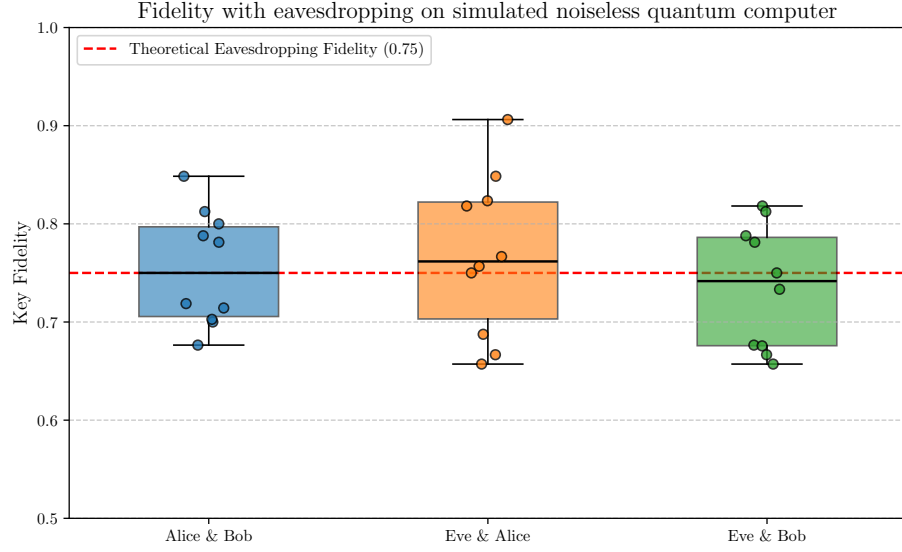


Figure 1: Box plot of the results obtained when eavesdropping on the circuit run on a simulated noiseless quantum computer.

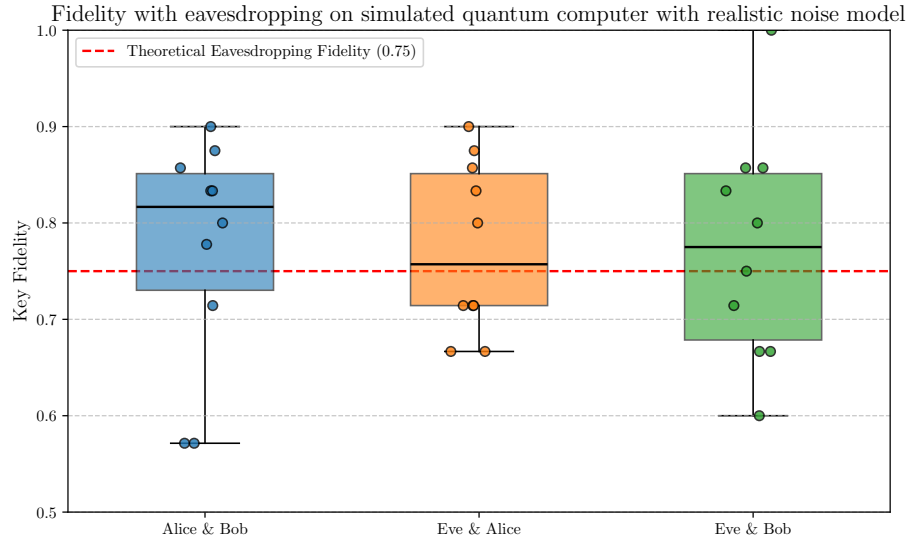


Figure 2: Box plot of the results obtained when eavesdropping on the circuit run on a simulated quantum computer with a realistic noise model.

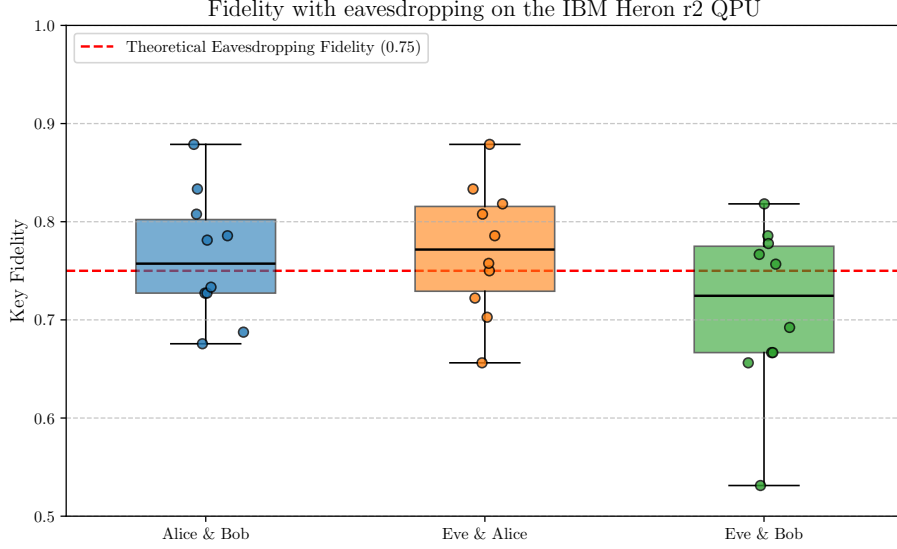


Figure 3: Box plot of the results obtained when eavesdropping on the circuit run on the IBM Heron r2 QPU.

	Noiseless offline		Noisy offline		Online	
	<i>fidelity</i>	<i>deviation</i>	<i>fidelity</i>	<i>deviation</i>	<i>fidelity</i>	<i>deviation</i>
Alice & Bob	1.0000	0.0000	1.0000	0.0000	0.9942	0.0093

Table 4: The mean fidelity and mean deviation obtained on different runs of the implemented protocol without eavesdropping.

implemented protocol. As proved by the corner case in fig. 2, there is a non-zero possibility that Eve obtains a key that matches the one of Bob’s, or Alice’s.

However, it should also be considered, that due to hardware limitations, none of the keys produced in any of the scenarios were longer than 32 bits, thus allowing for an encryption of 8 ASCII characters, if the key is used in a one-time pad. Furthermore, in the corner case evident in fig. 2, the key was only 7 bits long, again due to hardware limitations. The length of key greatly influences the possibility of eve actually *being lucky* and guessing the right key.

However, as is evident from the results, the strength of the BB84 protocol is not that it is perfectly secure no matter the type of attack, but rather that active attacks can be reliably detected, when considering long keys.

6 Conclusion

The objective for the mini-project was to implement the BB84-Protocol in order to understand how quantum mechanics can be used for secure communica-

tions. The BB84-protocol has been introduced, and the underlying properties of quantum mechanics enforcing the theoretic security of the protocol was briefly touched upon.

The protocol was implemented in Python as a means of constructing a key. In the implementation, two separate threads communicated over a socket thread in order to simulate communication between different parties au lieu of access to an actual quantum channel. The implementation used the Qiskit SDK [8] to map the produced code to machine level instructions runnable on actual quantum hardware provided access to by IBM. Although hardware limitations limited the obtained key to a maximum of 32 bits for each iteration of the protocol, the fundamental principles of the protocol were verified through the experiments on both simulators and the Heron r2 QPU. All the results were in line with the theory and eavesdropping on the circuit was coupled with increased error rates that allowed for the detection of said eavesdropping.

Acknowledgments

The Python scripts used to produce the figures and extract the relevant data for analysis of results in section 5 was created with the help of Google Gemini. Additionally, Google Gemini was used to format and transcribe the exact data into the tables presented in sections 4 and 5 provided by the author in accordance to a template also provided by the author.

A Output of implementation with eavesdropping

The output shown below was obtained by running the CLI with the following command: `python main.py --noiseless --with-eve --key-length 8 -O-1`. Please note that the circuit was run on a simulation of an ideal quantum computer with the only noise in the system being introduced by the eavesdropper.

```
1 Running circuit on local simulator
2 num qubits: 32
3 [01:19:27,249] Alice: Preparing states
4 [01:19:27,250] Alice: Transmitting qubits through quantum channel
5 [01:19:27,250] Alice: Announced message on classic channel
6     --- Message ---
7     Sender:    Alice
8     Time:      1
9     Header:    request ok announcement
10    Content:    states received
11    -----
12    --- Person ---
13    bits:       [0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1,
14    ↪ 1, 1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1]
15    bases:      [1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1,
16    ↪ 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1]
17    key:        []
18    -----
19 [01:19:27,250] Eve: Listening on socket.
20 [01:19:27,250] Eve: Stealing message from classic channel
21 [01:19:27,250] Eve: Receiving qubits through quantum channel
22 [01:19:27,250] Eve: Measuring states
23 [01:19:27,251] Alice: Listening on socket.
24 [01:19:27,251] Alice: Waiting to receive message...
25 [01:19:27,251] Bob: Listening on socket.
26 [01:19:27,251] Bob: Waiting to receive message...
27 [01:19:27,253] Eve: Preparing states
28 [01:19:27,253] Eve: Transmitting qubits through quantum channel
29 [01:19:27,253] Eve: Waiting to receive message...
30 [01:19:27,354] Eve: Waiting to receive message...
31 [01:19:27,451] Alice: Waiting to receive message...
32 [01:19:27,451] Bob: Receiving qubits through quantum channel
33 [01:19:27,452] Bob: Measuring states
34 [01:19:27,454] Bob: Announced message on classic channel
35     --- Message ---
36     Sender:    Bob
37     Time:      3
38     Header:    confirm ok announcement
39     Content:    states received
40     -----
```

```

39      --- Person ---
40      bits:      [0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1,
41      ↪      1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1]
42      bases:     [1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0,
43      ↪      0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1]
44      key:       []
45      -----
46      [01:19:27,454] Eve: Waiting to receive message...
47      [01:19:27,454] Bob: Waiting to receive message...
48      [01:19:27,554] Eve: Waiting to receive message...
49      [01:19:27,651] Alice: Announced message on classic channel
50      --- Message ---
51      Sender:     Alice
52      Time:       5
53      Header:     base announcement
54      Content:    [1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1,
55      ↪      1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1]
56      -----
57      --- Person ---
58      bits:      [0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1,
59      ↪      1, 1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1]
60      bases:     [1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1,
61      ↪      1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1]
62      key:       [1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1,
63      ↪      1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1]
64      -----
65      [01:19:27,651] Alice: Waiting to receive message...
66      [01:19:27,654] Bob: Reading bases retrieved through classic channel
67      [01:19:27,654] Bob: Discarding bit and base indices: [10, 15, 16, 17, 18,
68      ↪      22, 26, 28, 29]
69      [01:19:27,654] Bob: Announced message on classic channel
70      --- Message ---
71      Sender:     Bob
72      Time:       7
73      Header:     discarded bits announcement
74      Content:    [10, 15, 16, 17, 18, 22, 26, 28, 29]
75      -----
76      --- Person ---
77      bits:      [0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0,
78      ↪      0, 0, 0, 0, 1]
79      bases:     [1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1,
80      ↪      0, 1, 0, 0, 1]
81      key:       [0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0,
82      ↪      0, 0, 0, 0, 1]
83      -----
84      [01:19:27,654] Eve: Reading discarded bits through classic channel
85      [01:19:27,654] Bob: Waiting to receive message...
86      [01:19:27,654] Eve: Discarding bit and base indices: [10, 15, 16, 17, 18,
87      ↪      22, 26, 28, 29]
88      [01:19:27,654] Eve: Discarding indices in key: [0, 1, 2, 3, 4, 5, 6, 7,
89      ↪      8, 9, 10]

```



```

78 [01:19:27,654] Eve: Waiting to receive message...
79 [01:19:27,754] Eve: Waiting to receive message...
80 [01:19:27,851] Alice: Reading discarded bits through classic channel
81 [01:19:27,851] Alice: Discarding bit and base indices: [10, 15, 16, 17,
    ↪ 18, 22, 26, 28, 29]
82 [01:19:27,852] Alice: Discarding indices in key: [0, 1, 2, 3, 4, 5, 6, 7,
    ↪ 8, 9, 10]
83 [01:19:27,852] Alice: Announced message on classic channel
84     --- Message ---
85     Sender:    Alice
86     Time:      9
87     Header:    bit check indices announcement
88     Content:   [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
89     -----
90     --- Person ---
91     bits:      [0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0,
    ↪ 0, 1, 0, 1, 1]
92     bases:     [1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1,
    ↪ 0, 1, 0, 0, 1]
93     key:       [0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1]
94     -----
95 [01:19:27,852] Alice: Waiting to receive message...
96 [01:19:27,854] Bob: Reading bit check through classic channel
97 [01:19:27,854] Bob: Discarding indices in key: [0, 1, 2, 3, 4, 5, 6, 7,
    ↪ 8, 9, 10]
98 [01:19:27,854] Bob: Announced message on classic channel
99     --- Message ---
100    Sender:    Bob
101    Time:      11
102    Header:    test bits announcement
103    Content:   [0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1]
104    -----
105    --- Person ---
106    bits:      [0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0,
    ↪ 0, 0, 0, 0, 1]
107    bases:     [1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1,
    ↪ 0, 1, 0, 0, 1]
108    key:       [0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1]
109    -----
110 [01:19:27,854] Bob: Waiting to receive message...
111 [01:19:27,855] Eve: Reading test bits through classic channel
112 [01:19:27,855] Eve: Waiting to receive message...
113 [01:19:27,955] Eve: Waiting to receive message...
114 [01:19:28,52] Alice: Reading test bits through classic channel
115 [01:19:28,52] Alice: Announced message on classic channel
116     --- Message ---
117     Sender:    Alice
118     Time:      13
119     Header:    request ok announcement
120     Content:   loss: 0.18181818181818177
121     -----

```

```

122     --- Person ---
123     bits:    [0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0,
124             ↪ 0, 1, 0, 1, 1]
125     bases:   [1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1,
126             ↪ 0, 1, 0, 0, 1]
127     key:     [0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1]
128     -----
129 [01:19:28,52] Alice: Waiting to receive message...
130 [01:19:28,55] Bob: Matched key: 'loss', value: '0.181818181818177'
131 [01:19:28,55] Bob: Announced message on classic channel
132     --- Message ---
133     Sender:   Bob
134     Time:     15
135     Header:   abort protocol
136     Content:  loss: 0.181818181818177
137     -----
138     --- Person ---
139     bits:    [0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0,
140             ↪ 0, 0, 0, 0, 1]
141     bases:   [1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1,
142             ↪ 0, 1, 0, 0, 1]
143     key:     [0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1]
144     -----
145 [01:19:28,55] Bob: Received stop flag, shutting down.
146 [01:19:28,252] Alice: Received stop flag, shutting down.
147 Alice's key: [0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1]
148 Bob's key:   [0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1]
149 Key length:  12
150 Fidelity:    0.8333333333333334
151 Loss:        0.16666666666666663
152     -----
153 Eve's key: [0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1]
154 Alice's key: [0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1]
155 Key length:  12
156 Fidelity:    0.6666666666666666
157 Loss:        0.3333333333333337
158     -----
159 Eve's key: [0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1]
160 Bob's key:  [0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1]
161 Key length:  12
162 Fidelity:    0.8333333333333334
163 Loss:        0.16666666666666663
164     -----

```

References

- [1] Michael A. Nielsen; Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010. ISBN: 978-1-107-00217-3.
- [2] Valerio Scarani et al. “The security of practical quantum key distribution”. In: *Reviews of Modern Physics* 81.3 (2009). ISSN: 1539-0756. DOI: 10.1103/revmodphys.81.1301. URL: <http://dx.doi.org/10.1103/RevModPhys.81.1301>.
- [3] Chris Bernhardt. *Quantum Computing for Everyone*. Massachusetts Institute of Technology, 2020. ISBN: 9780262539531.
- [4] IBM and Qiskit contributors. *Measure qubits*. URL: <https://quantum.cloud.ibm.com/docs/en/guides/measure-qubits> (visited on 05/28/2026).
- [5] Víctor Zapatero, Álvaro Navarrete, and Marcos Curty. “Implementation Security in Quantum Key Distribution”. eng. In: *Advanced Quantum Technologies* 8.2 (2025). ISSN: 2511-9044.
- [6] Minming Geng. “Advances of Quantum Key Distribution and Network Nonlocality”. eng. In: *Entropy (Basel, Switzerland)* 27.9 (2025), pp. 950–. ISSN: 1099-4300.
- [7] Patrick Andriolo et al. “Quantum Key Distribution with Imperfections: Recent Advances in Security Proofs”. eng. In: *Brazilian journal of physics* 56.4 (2026), pp. 157–. ISSN: 0103-9733.
- [8] Ali Javadi-Abhari et al. *Quantum computing with Qiskit*. 2024. DOI: 10.48550/arXiv.2405.08810. arXiv: 2405.08810 [quant-ph].
- [9] IBM and Qiskit contributors. *Aer - high performance quantum circuit simulation for Qiskit*. URL: <https://github.com/Qiskit/qiskit-aer> (visited on 05/30/2026).
- [10] IBM and Qiskit contributors. *Introduction to transpilation*. URL: <https://quantum.cloud.ibm.com/docs/en/guides/transpile> (visited on 05/30/2026).